Software Architecture and Design Specification

Project: Task Manager Application

Team: Team 6

Version: 1.0

Authors: R Suhaas; Rachel Sara Mathew; Rachepalli Nandhitha; Rachith M

Date: 04-10-2026

Status: Final

This specification defines the Version 1.0 target architecture and detailed design for the Team 6 Task Manager Application. It maps the planned authentication, task, collaboration, notification, reporting, performance, and security requirements to a modular web architecture that can be implemented and tested incrementally.

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Team 6	Initial software architecture and design specification for submission

Approvals

Role	Name	Registration Number	Signature and Date
Team Member	R Suhaas	PES2UG24CS389	_____
Team Member	Rachel Sara Mathew	PES2UG24CS390	_____
Team Member	Rachepalli Nandhitha	PES2UG24CS391	_____
Team Member	Rachith M	PES2UG24CS392	_____
Faculty or Evaluator			_____

1 Introduction

1.1 Purpose

This document specifies how the Task Manager Application is divided into components, how those components communicate, how data is stored, and how the design satisfies the project requirements. It is the implementation guide for developers and the structural reference for testing, security review, maintenance, and evaluation.

1.2 Scope

Version 1.0 covers user registration and authentication, password protection, task creation and lifecycle management, filtering and search, projects and boards, team membership and assignment, comments, reminders and notifications, dashboards and reports, audit-oriented logging, and protected REST APIs. Payment processing and third-party calendar synchronization are outside the Version 1.0 boundary.

1.3 Audience

The intended readers are Team 6 developers, testers, faculty evaluators, security reviewers, and future maintainers. Developers should use Sections 3 and 4 to align implementation choices; testers should use the requirement traceability and interface definitions to derive integration and system tests.

1.4 Definitions

Term	Meaning
API	Application Programming Interface
CRUD	Create, read, update, and delete operations
JWT	JSON Web Token used to represent an authenticated session
RBAC	Role-based access control
REST	Representational State Transfer
SPA	Single-page web application
TLS	Transport Layer Security
UML	Unified Modeling Language

2 Document Overview

2.1 How to Use This Document

Section 3 describes the architectural drivers, stakeholders, components, technology choices, risks, requirement allocation, and security controls. Section 4 provides interaction flows, REST interfaces, data design, error handling, monitoring, user-experience decisions, and implementation priorities. The appendices summarize terms, references, tools, and design assumptions.

2.2 Related Documents

- Task Manager Application Software Test Plan, Team 6, Version 1.0, dated 16-09-2026.
- Task Manager Application Software Requirements Specification, Version 1.0, as referenced by the test plan.
- Software Engineering Mini-Project Deliverables and faculty submission instructions.
- GitHub repository: https://github.com/Nandhitha189/Task_Manager_Mini_Project

3 Architecture

3.1 Goals and Constraints

The target design is driven by the following goals:

- Keep private tasks and project data isolated by authenticated user and authorized project membership.
- Return at least 90 percent of normal page and API requests within two seconds for a test load of up to 500 concurrent users.

- Support later horizontal scaling toward 5,000 concurrent users without changing application-domain code.
- Provide a responsive interface from 360-pixel mobile displays through 1920-pixel desktop displays.
- Keep modules separable so the four-person team can develop and test features in parallel.

The main constraints are the mini-project schedule, a small development team, the need to demonstrate all critical flows locally, dependence on an external email service for password reset and reminders, and the absence of payment and calendar integrations from Version 1.0.

3.2 Stakeholders and Concerns

Stakeholder	Primary Concerns	Architectural Response
End users	Fast, clear task management; privacy; dependable reminders	Responsive React UI, ownership checks, indexed queries, background notification jobs
Project owners	Membership, assignment, comments, board visibility	Project and membership domain model with project-scoped authorization
Administrators	Aggregate usage and operational visibility	Role-protected analytics endpoints and structured operational metrics
Developers	Modularity, testability, manageable deployment	Layered modular monolith with dependency boundaries and automated tests
Testers	Stable interfaces and traceable behavior	Versioned REST contract, consistent errors, requirement-to-component mapping
Faculty evaluators	Complete design evidence and feasible implementation	UML views, API definitions, risk analysis, and explicit design decisions

3.3 Component UML Diagram

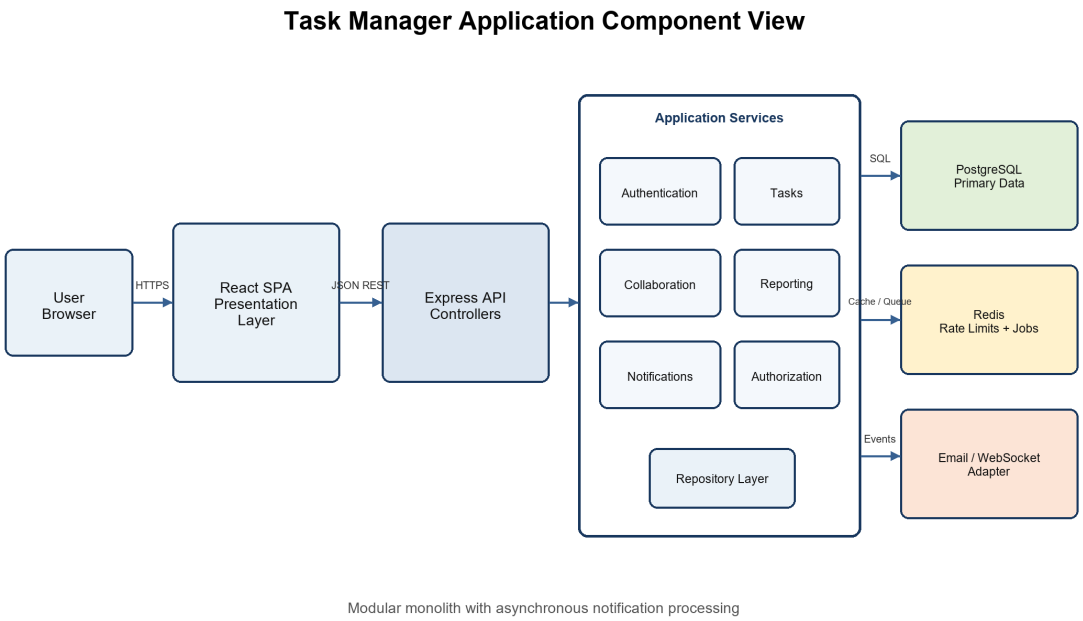


Figure 1 Task Manager Application component view

3.4 Component Descriptions

Component	Responsibility	Key Dependencies
React SPA	Presents authentication, boards, task forms, filters, dashboards, and responsive navigation; stores only short-lived client state.	REST API; browser storage policy
Express API Controllers	Validate transport-level input, authenticate tokens, call application services, and serialize responses.	Authentication middleware; application services
Authentication Service	Registers users, verifies salted password hashes, issues and refreshes JWTs, initiates password resets, and enforces lockout rules.	User repository; Redis; email adapter
Task Service	Implements task CRUD, status transitions, priorities, tags, filtering, search, and due-date rules.	Task repository; authorization service
Collaboration Service	Manages projects, memberships, task assignment, and comments.	Project repository; notification service
Authorization Service	Applies ownership, project membership, assignee, and administrative policies on every protected operation.	Authenticated principal; repositories
Notification Service	Consumes domain events and sends in-app, email, or WebSocket notifications asynchronously.	Redis job queue; email or real-time adapter
Reporting Service	Computes status counts, weekly productivity measures, and authorized aggregate analytics.	Task repository; project repository
Repository Layer	Maps domain operations to parameterized PostgreSQL queries or an ORM and provides transaction boundaries.	PostgreSQL
Observability Layer	Produces structured logs, request correlation, health checks, metrics, and audit events without exposing secrets.	Application modules; monitoring sink

3.5 Chosen Architecture Pattern and Rationale

Version 1.0 uses a layered modular monolith. The presentation layer is a React SPA; the interface layer is a versioned Express REST API; the application layer contains authentication, task, collaboration, notification, reporting, and authorization modules; and the infrastructure layer contains repositories, Redis integration, external adapters, logging, and configuration.

This pattern provides clear ownership and test boundaries without the deployment and distributed-transaction cost of microservices. Modules communicate through explicit service interfaces and domain events, which leaves a controlled migration path if notification or reporting

workloads later justify separate services. A single deployable backend and one relational database are appropriate for the team size and submission schedule.

3.6 Technology Stack and Data Stores

Area	Selected Technology	Rationale
Frontend	React with JavaScript, HTML, and CSS	Matches the test plan, supports reusable components and responsive SPA behavior
Backend	Node.js with Express	Keeps one primary language across client and server and supports a lightweight modular REST API
Primary database	PostgreSQL	Provides transactions, relational constraints, indexing, and full-text search options
Transient store	Redis	Supports authentication rate limits, short-lived reset tokens, lockout counters, and background jobs
Authentication	JWT access tokens with rotating refresh tokens	Supports stateless API authorization while permitting server-side revocation of refresh sessions
Password protection	bcrypt or Argon2id	Provides salted one-way password hashing and tunable work factors
Notifications	Email provider plus optional WebSocket adapter	Separates external delivery from domain logic and allows mock providers in testing
Deployment	Reverse proxy, stateless API instances, worker process, PostgreSQL, and Redis	TLS termination and independently scalable web and worker processes
Testing	Jest, React Testing Library, Supertest, Postman, OWASP ZAP	Covers unit, component, API, integration, and security testing

All environment-specific values, including database credentials, token signing keys, email credentials, and allowed origins, are injected through validated configuration. Secrets are never committed to source control. Production traffic enters through an HTTPS reverse proxy using TLS 1.2 or higher; the API and worker remain stateless so additional instances can be added behind a load balancer.

3.7 Risks and Mitigations

Risk	Impact	Mitigation
Frontend and backend contract drift	Integration failures near submission	Maintain one versioned OpenAPI contract and run request or response schema tests in continuous integration
Unauthorized cross-user access	Disclosure or modification of private tasks	Centralize policy checks, use ownership-filtered queries, and execute negative authorization tests
Email provider unavailable	Password-reset or reminder delays	Queue delivery attempts, retry with backoff, expose in-app notifications, and use a mock provider for tests
Slow dashboard queries	Response-time target missed	Add indexes, pre-aggregate expensive metrics, paginate lists, and inspect query plans
Duplicate background jobs	Repeated notifications	Use idempotency keys and record delivery status per event and recipient
Loss of a team member or schedule slippage	Incomplete lower-priority features	Prioritize authentication and task CRUD, define module owners, and integrate continuously
Secret or personal data in logs	Security and privacy exposure	Apply field allowlists, redact authorization headers and passwords, and restrict log access

3.8 Traceability to Requirements

Requirement IDs	Capability	Owning Components
TM-F-001 to TM-F-005	Registration, login, password handling, reset, and lockout	React SPA; Authentication Service; User Repository; Redis; Email Adapter
TM-F-010 to TM-F-015	Task CRUD, status, priority, and tags	Task Service; Authorization Service; Task Repository
TM-F-020 to TM-F-023	Filter, sort, search, projects, and boards	Task Service; Collaboration Service; PostgreSQL indexes
TM-F-030 to TM-F-032	Due, overdue, and assignment notifications	Notification Service; Worker; Redis; delivery adapters
TM-F-040 to TM-F-042	Invitations, assignments, and comments	Collaboration Service; Authorization Service; Notification Service
TM-F-050 to TM-F-052	Dashboards, weekly reports, and admin analytics	Reporting Service; Authorization Service; repositories
TM-NF-001 to TM-NF-005	Performance, reliability, security, usability, and scalability	Reverse proxy; stateless API; database indexes; observability; responsive SPA
TM-SR-001 to TM-SR-005	TLS, hashing, input validation, RBAC, and rate limiting	Reverse proxy; Authentication Service; validation middleware; Authorization Service; Redis

3.9 Security Architecture

Every protected endpoint authenticates the caller and then authorizes the requested resource. The API rejects client-supplied ownership fields, derives the acting user from the validated token, and scopes repository queries by owner or project membership. Passwords are salted and hashed; reset links use short-lived single-use tokens; refresh sessions can be revoked; authentication attempts are rate-limited to 10 requests per minute per IP; and five consecutive failed logins trigger a 15-minute account lock.

STRIDE Threat	Example	Primary Controls
Spoofing	Stolen credentials or forged tokens	Strong password hashing, signed short-lived access tokens, refresh rotation, login rate limits
Tampering	Changing another user's task ID or request body	Ownership-filtered queries, project membership checks, validation, transactions, audit events
Repudiation	User denies assigning or deleting a task	Correlation IDs and append-only audit records for security-sensitive operations
Information disclosure	Tasks exposed through APIs or logs	TLS 1.2+, least-privilege queries, safe error messages, log redaction, restricted CORS
Denial of service	Repeated login, search, or report requests	Redis rate limits, pagination, request-size limits, query timeouts, health monitoring

STRIDE Threat	Example	Primary Controls
Elevation of privilege	Member invokes admin analytics or project-owner actions	Central policy functions, explicit roles, default deny, authorization regression tests

4 Design

4.1 Design Overview

The domain model centers on users, projects, memberships, tasks, tags, comments, notification events, refresh sessions, and audit events. Controllers remain thin: they validate input and call application services. Services enforce workflows and authorization before repositories access data. External providers are reached only through adapters so tests can replace them with deterministic fakes.

The design applies the following rules consistently:

- Every API resource uses an opaque identifier and includes created and updated timestamps.
- Write operations validate the complete command and execute related database changes within a transaction.
- Task status is restricted to To Do, In Progress, or Completed; priority is restricted to Low, Medium, or High.
- Lists are paginated and use stable ordering. Search and filters are combined through parameterized queries.
- Domain events are recorded after a successful write and processed asynchronously for notification delivery.

4.2 UML Sequence Diagrams

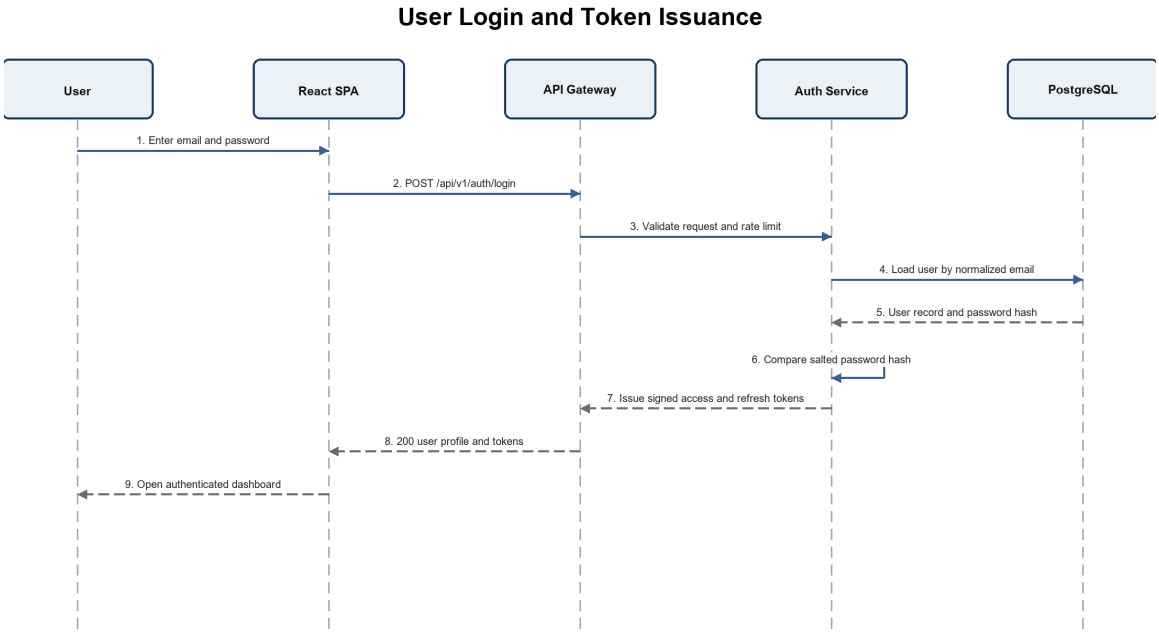


Figure 2 Login flow with password verification and token issuance

The login flow applies transport validation and an IP-based rate limit before credential verification. The Authentication Service loads one user record, compares the stored salted hash, updates the failed-attempt or lockout state as required, and returns short-lived access credentials only after a successful comparison.

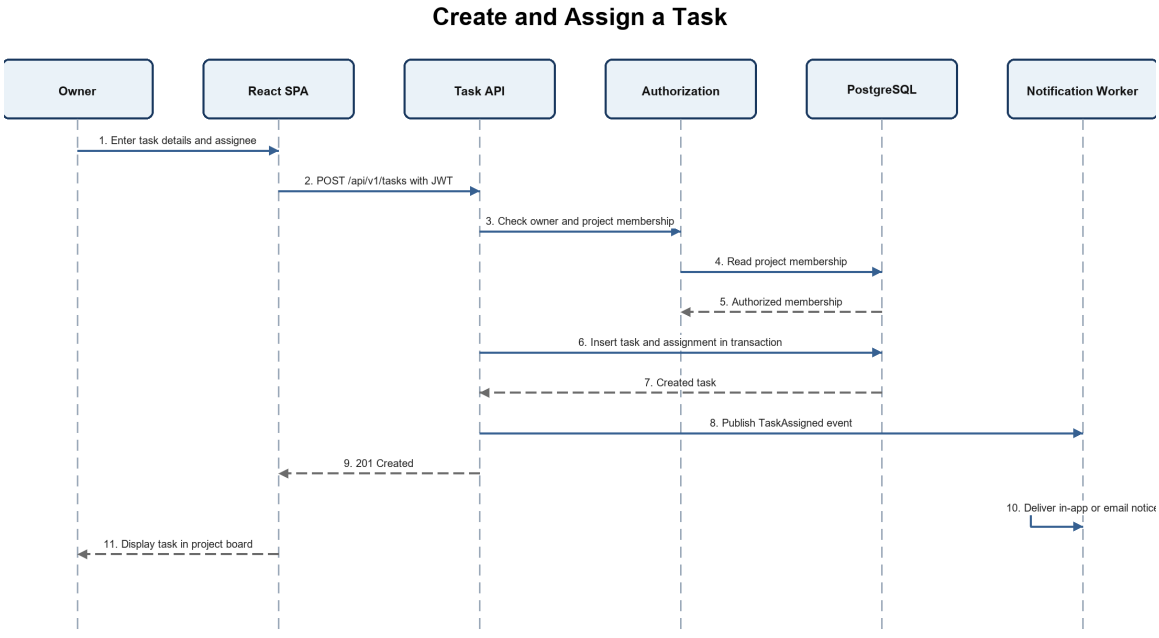


Figure 3 Create and assign task flow with asynchronous notification

The task flow checks project membership before the database write. Task creation and assignment are committed together; only then is a TaskAssigned event queued. Notification delivery therefore does not delay the API response and can be retried without duplicating the task.

4.3 API Design

All endpoints use the prefix /api/v1, accept and return JSON, and require Authorization: Bearer <token> unless marked public. Dates and times use ISO 8601 UTC. Collection responses include items, page, pageSize, and total. Mutation endpoints return the created or updated representation.

Method and Path	Authorization	Purpose and Main Result
POST /auth/register	Public	Create a user from name, email, and password; return 201 without exposing the password hash
POST /auth/login	Public; 10 requests/minute/IP	Validate credentials and return access token, refresh token, and user profile
POST /auth/refresh	Valid refresh session	Rotate the refresh token and issue a new access token
POST /auth/password-reset/request	Public	Create a short-lived reset token and enqueue an email
GET /tasks	Authenticated	List only authorized tasks with status, priority, category, due-date, project, and search filters
POST /tasks	Project owner or authorized member	Create a task and optional assignment; return 201
PATCH /tasks/{taskId}	Owner or permitted assignee	Update editable fields or allowed status transition
DELETE /tasks/{taskId}	Task owner or project owner	Delete after client confirmation; return 204
POST /projects/{projectId}/members	Project owner	Invite or add a registered user to the project
POST /tasks/{taskId}/comments	Project member	Add a comment and enqueue participant notifications
GET /dashboard	Authenticated	Return status counts and weekly created-versus-completed measures
GET /admin/analytics	Administrator	Return authorized aggregate usage analytics

4.3.1 Interface Definition for Authentication

Item	Definition
Request	POST /api/v1/auth/login with {email, password}
Success	200 with {user: {id, name, email, role}, accessToken, refreshToken, expiresIn}
Validation error	400 with field-level validation details
Authentication failure	401 with a generic invalid-credentials message
Account locked	423 with retryAfterSeconds; no password details
Rate limited	429 with retryAfterSeconds after the tenth request in one minute from the same IP

4.3.2 Interface Definition for Tasks

Item	Definition
Request	POST /api/v1/tasks with {projectId, title, description, dueDate, priority, tagIds, assigneeId}
Success	201 with the created task, assignment summary, tags, and timestamps
Validation error	400 when required fields, enum values, or due-date formats are invalid
Unauthenticated	401 when no valid access token is supplied
Forbidden	403 when the caller lacks project membership or assignment permission
Conflict	409 for a conflicting idempotency key or invalid concurrent state transition

4.4 Data Design

Entity	Important Fields	Relationships and Constraints
User	id, name, normalized_email, password_hash, role, failed_login_count, locked_until	Unique normalized email; password hash never returned by API
Project	id, owner_id, name, description, created_at	Owner references User; project name required
ProjectMember	project_id, user_id, membership_role, joined_at	Composite unique key; role limited to owner or member
Task	id, project_id, owner_id, assignee_id, title, description, status, priority, due_at, completed_at	Status and priority checks; indexes on owner, assignee, project, status, priority, and due date
Tag	id, owner_id or project_id, name	Unique within its owning scope
TaskTag	task_id, tag_id	Composite unique key; cascading removal with task
Comment	id, task_id, author_id, body, created_at	Author must be an authorized project member
Notification	id, recipient_id, type, payload, read_at, delivery_status	Indexed by recipient and creation time; idempotency key prevents duplicates
RefreshSession	id, user_id, token_hash, expires_at, revoked_at	Stores a token hash, not the raw refresh token
AuditEvent	id, actor_id, action, resource_type, resource_id, correlation_id, occurred_at	Append-only; excludes secrets and full sensitive payloads

Foreign keys protect referential integrity, and database constraints enforce enumerated values and required relationships. Task searches use normalized text and indexes appropriate to the selected PostgreSQL search strategy. Reporting queries operate on indexed timestamps and status fields; expensive aggregate views may be cached briefly in Redis without treating the cache as the system of record.

4.5 Error Handling Logging and Monitoring

The API returns a consistent error object with code, message, optional field-level details, and a correlationId. Messages presented to users explain the corrective action but do not reveal password state, database structure, stack traces, or internal identifiers. Expected client errors use 400, 401, 403, 404, 409, 423, or 429; unexpected failures return 500 and are recorded with the correlation ID.

Operational Signal	Purpose	Example Threshold or Action
HTTP latency percentiles	Validate the two-second response-time objective	Investigate when p90 exceeds two seconds during normal-load testing
5xx error rate	Detect application or dependency failure	Alert on sustained increases and inspect correlation IDs
Authentication failures and 429 responses	Detect misuse and validate controls	Review spikes by endpoint and IP without logging passwords

Operational Signal	Purpose	Example Threshold or Action
Database pool and slow queries	Prevent resource exhaustion and dashboard delays	Capture query duration and analyze indexed execution plans
Notification queue age and failures	Detect delayed reminders or assignments	Retry with bounded exponential backoff and move permanent failures to a review queue
Health and readiness checks	Keep traffic away from unhealthy instances	Verify API process, database connectivity, and required configuration

4.6 UX Design

- Use a responsive layout that preserves the primary create, filter, search, and status-update actions on mobile and desktop displays.
- Present tasks consistently as list or board cards showing title, status, priority, due date, project, and assignee without requiring a detail view.
- Provide keyboard-visible focus, semantic labels, sufficient color contrast, readable error text, and text labels in addition to status colors.
- Confirm destructive deletion and explain the affected task before the final action.
- Use optimistic updates only for reversible low-risk actions; restore the previous state and show a clear message if the server rejects the change.
- Display empty, loading, success, validation, offline, and authorization states explicitly so users are not left with a blank screen.

4.7 Open Issues and Next Steps

1. Confirm the final backend implementation stack and commit the application source beside the project documents.
2. Publish the authoritative SRS and derive an OpenAPI file from the approved interface definitions.
3. Select the email provider, hosting environment, Redis deployment, and production secret-management mechanism.
4. Implement the database migrations and add automated unit, API, authorization, and integration tests for the critical flows.
5. Measure the 500-user normal-load target, validate the 5,000-user scaling design, and record the actual capacity results.
6. Complete a security review with dependency scanning, OWASP ZAP, rate-limit checks, and explicit cross-user access tests.

5 Appendices

5.1 Glossary

Term	Project-Specific Meaning
Access token	Short-lived signed JWT presented to protected API endpoints
Domain event	A committed fact such as TaskAssigned that triggers asynchronous work
Modular monolith	One backend deployment whose internal modules have explicit responsibilities and dependency boundaries
Owner	User with authority to manage a personal task or a project's membership and tasks
Worker	Background process that consumes queued jobs such as notification delivery

5.2 References

- Team 6, Task Manager Application Software Test Plan, Version 1.0, 16-09-2026.
- OWASP Application Security Verification Standard and OWASP API Security Top 10.
- IEEE/ISO/IEC 42010, Systems and software engineering - Architecture description.
- PostgreSQL, Node.js, Express, React, Redis, and JSON Web Token official documentation.
- GitHub project repository: https://github.com/Nandhitha189/Task_Manager_Mini_Project

5.3 Tools

Recommended tools are Visual Studio Code for development, GitHub for version control, an OpenAPI editor for interface contracts, Postman for exploratory API testing, Jest and Supertest for automated backend tests, React Testing Library for UI behavior, OWASP ZAP for dynamic security testing, and a load-testing tool such as k6 for performance measurements.

5.4 Design Assumptions

- Version 1.0 is a web application accessed through modern desktop and mobile browsers.
- A user may own personal tasks and may also participate in projects with other registered users.
- The relational database is the authoritative data store; Redis contains only reconstructable or short-lived state.
- Payment processing and third-party calendar synchronization remain outside the release boundary.
- External email and real-time delivery are accessed through adapters so the project can run with test doubles.